QUE 1:

CODE:

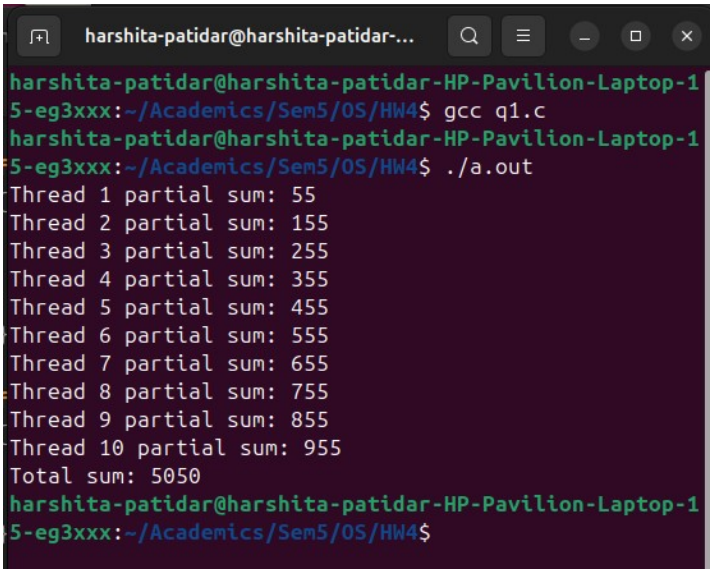
```
#include<pthread.h>
#include<stdio.h>
#include<stdlib.h>

#define SIZE 100
#define SEGMENT_SIZE 10
#define NUM_THREADS 11

typedef struct  {
   int *arr;
   int start;
} args;

void* partial_sum(void* arg)
{
    args* a=(args*) arg;
    int sum=0;

    for(int i=a->start; i<a->start + SEGMENT_SIZE; i++)
    {
       sum+=a->arr[i];
    }

    int *result=malloc(sizeof(int));
    *result=sum;
    return result;
}

void *final_sum(void* arg)
{
   int **partial=(int**)arg;
   int total=0;

   for(int i=0;i<10;i++)
      total+=*partial[i];

   int *result=malloc(sizeof(int));
   *result=total;
   return result;
}

int main() {
   int arr[SIZE];
   for(int i=0;i<SIZE;i++)
   {
      arr[i]=i+1;
   }
```

```
  pthread_t threads[NUM_THREADS];
  args arguments[10];
  int *partial_res[10];

  for(int i=0;i<10;i++)
  {
    arguments[i].arr=arr;
    arguments[i].start=i*SEGMENT_SIZE;
    pthread_create(&threads[i],NULL,partial_sum,&arguments[i]);
  }

  for(int i=0;i<10;i++)
  {
    pthread_join(threads[i],(void**)&partial_res[i]);
    printf("Thread %d partial sum: %d\n", i+1, *partial_res[i]);
  }

  pthread_create(&threads[10],NULL,final_sum,partial_res);

  int *total_sum;
  pthread_join(threads[10],(void**)&total_sum);

  printf("Total sum: %d\n", *total_sum);

  for(int i=0;i<10;i++)
  {
    free(partial_res[i]);
  }

  free(total_sum);


   return 0;
}
```

Observation:

The program uses threads to compute segment-wise sums of an array, and a final thread collects these results to calculate the total sum, demonstrating thread creation, joining, and result retrieval.

Screenshot:



QUE 2:

CODE:

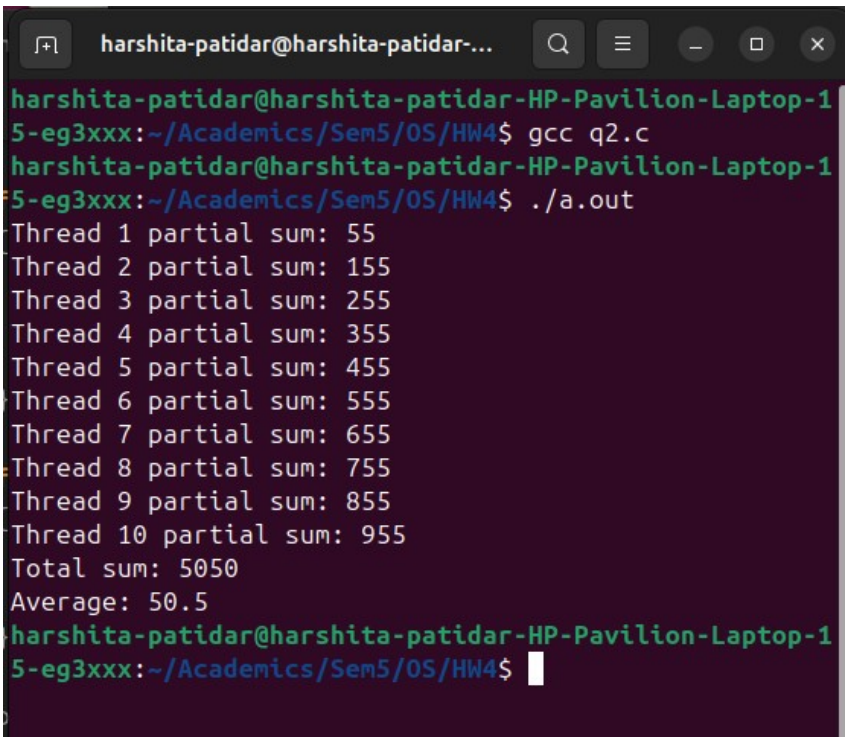
```
#include<pthread.h>
#include<stdio.h>
#include<stdlib.h>

#define SIZE 100
#define SEGMENT_SIZE 10
#define NUM_THREADS 11

typedef struct  {
   int *arr;
   int start;
} args;

void* partial_sum(void* arg)
{
    args* a=(args*) arg;
    int sum=0;

    for(int i=a->start; i<a->start + SEGMENT_SIZE; i++)
    {
       sum+=a->arr[i];
    }

    int *result=malloc(sizeof(int));
    *result=sum;
    return result;
}

void *final_sum(void* arg)
{
   int **partial=(int**)arg;
   int total=0;

   for(int i=0;i<10;i++)
      total+=*partial[i];

   int *result=malloc(sizeof(int));
   *result=total;
   return result;
}

int main() {
   int arr[SIZE];
   for(int i=0;i<SIZE;i++)
   {
```

```
    arr[i]=i+1;
  }

  pthread_t threads[NUM_THREADS];
  args arguments[10];
  int *partial_res[10];

  for(int i=0;i<10;i++)
  {
    arguments[i].arr=arr;
    arguments[i].start=i*SEGMENT_SIZE;
    pthread_create(&threads[i],NULL,partial_sum,&arguments[i]);
  }

  for(int i=0;i<10;i++)
  {
    pthread_join(threads[i],(void**)&partial_res[i]);
    printf("Thread %d partial sum: %d\n", i+1, *partial_res[i]);
  }

  pthread_create(&threads[10],NULL,final_sum,partial_res);

  int *total_sum;
  pthread_join(threads[10],(void**)&total_sum);

  printf("Total sum: %d\n", *total_sum);

  double avg=(double)(*total_sum)/SIZE;
  printf("Average: %.1f\n",avg);

  for(int i=0;i<10;i++)
  {
    free(partial_res[i]);
  }

  free(total_sum);


   return 0;
}
```

Observation:

The program demonstrates the use of threads where each of the first 10 threads computes the sum of a segment of an array, and the 11th thread sums these partial results to get the total. The main function then calculates the average, showing thread creation, joining, and result retrieval.

Screenshot:



QUE 4:

CODE:

```
#include<pthread.h>
#include<stdio.h>
#include<stdlib.h>

int SEGMENT_SIZE;
int N;

typedef struct  {
   int *arr;
   int start;
} args;

int max(int a, int b) {
    return (a > b) ? a : b;
}


void* segment_max(void* arg)
{
    args* a=(args*) arg;
    int maxi=a->arr[a->start];

    for(int i=a->start+1; i<a->start + SEGMENT_SIZE; i++)
    {
       maxi=max(maxi,a->arr[i]);
```

```
    }

    int *result=malloc(sizeof(int));
    *result=maxi;
    return result;
}

void *overall_max(void* arg)
{
  int **segmentMax=(int**)arg;
  int maxi=*segmentMax[0];

  for(int i=1;i<N;i++)
   {
     maxi=max(maxi,*segmentMax[i]);
   }

  int *result=malloc(sizeof(int));
  *result=maxi;
  return result;
}

int main() {
  int arr[] = { 3, 5 ,232, 2, 32, 48, 9, 12, 34, 56, 34, 89, 21, 7, 51, 17};
  N=4;
  int SIZE=sizeof(arr)/sizeof(arr[0]);
  SEGMENT_SIZE=SIZE/N;


  pthread_t threads[N+1];
  args arguments[N];
  int *segmentMax[N];

  for(int i=0;i<N;i++)
  {
    arguments[i].arr=arr;
    arguments[i].start=i*SEGMENT_SIZE;
    pthread_create(&threads[i],NULL,segment_max,&arguments[i]);
  }

  for(int i=0;i<N;i++)
  {
    pthread_join(threads[i],(void**)&segmentMax[i]);
    printf("Thread %d max: %d\n", i+1, *segmentMax[i]);
  }

  pthread_create(&threads[N],NULL,overall_max,segmentMax);

  int *maxi;
  pthread_join(threads[N],(void**)&maxi);

  printf("Overall Max: %d\n", *maxi);
```

```
  for(int i=0;i<N;i++)
  {
    free(segmentMax[i]);
  }

  free(maxi);


   return 0;
}
```

Observation:

The program demonstrates the use of threads where each of the first N threads finds the maximum value in its assigned segment of the array, and the final (N+1th) thread collects these segment maxima to determine the overall maximum. It illustrates thread creation, joining, result retrieval, and dynamic memory management in a multithreaded environment.

Screenshot: